

Capítulo 1 - Building Blocks

Pregunta 1

- **Respuesta correcta:** D, E
- **Explicación:**

La opción E es la firma canónica del método `main()`; necesitas memorizarla. La opción D es una forma alternativa válida con el modificador redundante `final`. La opción A es incorrecta porque el método `main()` debe ser `public`. Las opciones B y F son incorrectas porque el método `main()` debe tener un tipo de retorno `void`. La opción C es incorrecta porque el método `main()` debe ser `static`.

Pregunta 2

- **Respuesta correcta:** C, D, E
- **Explicación:**

Las sentencias `package` e `import` son ambas opcionales. Si ambas están presentes, el orden obligatorio debe ser: primero `package`, luego `import` y finalmente `class`. La opción A es incorrecta porque coloca `class` antes de `package` e `import`. La opción B es incorrecta porque coloca `import` antes de `package`. La opción F es incorrecta porque coloca `class` antes de `package`.

Pregunta 3

- **Respuesta correcta:** A, E
- **Explicación:**

`Bunny` es una clase, lo cual se puede apreciar a partir de su declaración: `public class Bunny`. La variable `bun` es una referencia a un objeto. El método `main()` es el punto de entrada estándar hacia un programa. La opción G es incorrecta porque lo que importa es el tipo del parámetro, no el nombre que se le asigne al parámetro.

Pregunta 4

- **Respuesta correcta:** B, E, G
- **Explicación:**

La opción A no es válida porque no se permite un único guion bajo aislado (`_`). La opción C no es un identificador válido porque `true` es una palabra reservada en Java. La opción D no es válida porque no se permite el punto (`.`) dentro de los identificadores. La opción F no es válida porque el primer carácter no es una letra, un signo de dólar (`$`) ni un guion bajo (`_`). Las opciones B, E y G son válidas ya que contienen únicamente caracteres permitidos.

Pregunta 5

- **Respuesta correcta:** A, D, F
- **Explicación:**

Nunca se garantiza la ejecución del recolector de basura (*Garbage Collection*), lo que convierte a la opción F en correcta y a la opción E en incorrecta. Seguido de esto, la clase compila y se ejecuta sin inconvenientes, por lo que la opción G es incorrecta. El objeto `Bear` creado en la línea 9 permanece accesible hasta la línea 13 mediante la variable de referencia `brownBear`, cumpliendo con la opción A. El objeto

Bear creado en la línea 10 es accesible a través de la referencia `polarBear` y también por `brownBear.pandaBear`. Tras la línea 12, el objeto todavía sigue siendo accesible mediante `brownBear.pandaBear`. Sin embargo, después de la línea 13, ya no es accesible debido a que `brownBear` ya no está accesible, lo cual convierte a la opción D en la respuesta final.

Pregunta 6

- **Respuesta correcta:** F
- **Explicación:**

Para resolver este problema, es necesario rastrear las llaves `{}` y ver cuándo las variables entran y salen de alcance (*scope*). Las variables de las líneas 2 y 7 solo están en alcance durante un bloque de una sola línea. La variable de la línea 12 solo está en alcance para el bucle `for`. Ninguna de estas está en alcance en la línea 14. Por el contrario, las tres variables de instancia de las líneas 3 y 4 están disponibles en todos los métodos de instancia. Además, las variables de las líneas 6, 9 y 10 están disponibles ya que el método y el bucle `while` todavía están dentro de su alcance. Esto da un total de 7 variables, lo que corresponde a la opción F.

Pregunta 7

- **Respuesta correcta:** C, E
- **Explicación:**

Lo primero que hay que reconocer es que se trata de un bloque de texto (*text block*) y el código dentro de `"""` es simplemente texto. Las opciones A y B son incorrectas porque las variables `numForks` y `numKnives` no se utilizan. Esto resulta conveniente ya que `numKnives` no está inicializada y no compilaría si se hiciera referencia a ella. La opción C es correcta ya que coincide con el texto. La opción D es incorrecta porque el bloque de texto no tiene una línea en blanco final. Finalmente, la opción E también es una respuesta ya que `" # knives` está indentado.

Pregunta 8

- **Respuesta correcta:** B, D, E, H
- **Explicación:**

Una variable `var` no puede inicializarse con un valor `null` sin un tipo explícito, pero sí se le puede asignar un valor `null` más adelante si el tipo subyacente no es primitivo. Por estas razones, la opción H es correcta, mientras que las opciones A y C son incorrectas. Las opciones B y D son correctas ya que los tipos subyacentes son `String` e `Integer`, respectivamente. La opción E es correcta ya que se trata de una expresión numérica válida. Aunque dividir entre cero produce una excepción en tiempo de ejecución, la pregunta se centraba únicamente en si el código compilaba. Por último, las opciones F y G son incorrectas porque `var` no se puede utilizar en declaraciones de asignación múltiple de variables.

Pregunta 9

- **Respuesta correcta:** C, E
- **Explicación:**

En los bloques de texto (*text blocks*), la línea de apertura con las tres comillas triples

""" debe ir seguida obligatoriamente por un salto de línea en su propia línea antes de que comience el contenido textual. Además, la presencia de una nueva línea al final del contenido depende directamente de si las comillas de cierre triples """ se colocan en una nueva línea separada o inmediatamente adyacentes al último carácter del texto.

Pregunta 10

- **Respuesta correcta:** A
- **Explicación:**

El compilador de Java (**javac**) toma los archivos fuente de código con extensión **.java** y genera archivos binarios con extensión **.class** que contienen el *bytecode* independiente del sistema operativo. Posteriormente, la Máquina Virtual de Java (JVM) ejecuta dicho *bytecode* utilizando el comando del entorno de ejecución **java**.

Pregunta 11

- **Respuesta correcta:** E
- **Explicación:**

Las dos primeras importaciones se pueden eliminar porque **java.lang** se importa de manera automática. Las dos siguientes importaciones se pueden eliminar debido a que **Tank** y **Water** están dentro del mismo paquete, haciendo que la opción correcta sea la E. Si **Tank** y **Water** estuvieran en paquetes diferentes, exactamente una de estas dos importaciones se podría remover. En ese caso, la respuesta habría sido la opción D.

Pregunta 12

- **Respuesta correcta:** A, C, D
- **Explicación:**

La línea 2 no compila puesto que solo se debe especificar un único tipo para la variable, haciendo que la opción A sea correcta. La línea 3 compila sin problema ya que declara una variable local dentro de un bloque inicializador de instancia que nunca llega a usarse. La línea 4 no compila porque Java no soporta la asignación de valores predeterminados (*default values*) a los parámetros de un método, haciendo que la opción C sea correcta. Por último, la línea 7 no compila debido a que la variable **fin** está dentro del alcance (*scope*) y es accesible únicamente dentro del inicializador de instancia de la línea 3, haciendo que la opción D sea correcta.

Pregunta 13

- **Respuesta correcta:** A, B, C
- **Explicación:**

La opción A es correcta porque importa todas las clases del paquete **aquarium**, incluyendo **aquarium.Water**. Las opciones B y C son correctas porque importan **Water** por el nombre de la clase. Dado que importar por el nombre de la clase tiene precedencia sobre los comodines, estas compilan. La opción D es incorrecta porque Java no sabe cuál de las dos clases **Water** que provienen de los comodines utilizar. La opción E es incorrecta porque no se puede especificar el mismo nombre de clase en dos importaciones.

Pregunta 11

- **Respuesta correcta:** E
- **Explicación:**

Las dos primeras importaciones se pueden eliminar porque `java.lang` se importa de manera automática. Las dos siguientes importaciones se pueden eliminar debido a que `Tank` y `Water` están dentro del mismo paquete, haciendo que la opción correcta sea la E. Si `Tank` y `Water` estuvieran en paquetes diferentes, exactamente una de estas dos importaciones se podría remover. En ese caso, la respuesta habría sido la opción D.

Pregunta 12

- **Respuesta correcta:** A, C, D
- **Explicación:**

La línea 2 no compila puesto que solo se debe especificar un único tipo, haciendo que la opción A sea correcta. La línea 3 compila sin problema ya que declara una variable local dentro de un bloque inicializador de instancia que nunca llega a usarse. La línea 4 no compila porque Java no soporta la asignación de valores predeterminados (*default values*) a los parámetros de un método, haciendo que la opción C sea correcta. Por último, la línea 7 no compila debido a que la variable `fins` está dentro del alcance (*scope*) y es accesible únicamente dentro del inicializador de instancia de la línea 3, haciendo que la opción D sea correcta.

Pregunta 13

- **Respuesta correcta:** A, B, C
- **Explicación:**

La opción A es correcta porque importa todas las clases del paquete `aquarium`, incluyendo `aquarium.Water`. Las opciones B y C son correctas porque importan `Water` por el nombre de la clase. Dado que importar por el nombre de la clase tiene precedencia sobre los comodines, estas compilan. La opción D es incorrecta porque Java no sabe cuál de las dos clases `Water` que provienen de los comodines utilizar. La opción E es incorrecta porque no se puede especificar el mismo nombre de clase en dos importaciones.

Pregunta 14

- **Respuesta correcta:** A, B, D, E
- **Explicación:**

La línea 3 no compila porque el sufijo `L` convierte el valor literal en un `long`, el cual no puede almacenarse directamente dentro de un `short`, haciendo que la opción A sea correcta. La línea 4 no compila porque `int` es un tipo entero, pero `2.0` es un valor literal `double`, haciendo que la opción B sea correcta. La línea 5 compila sin problemas. Las líneas 6 y 7 no compilan porque `numPets` y `numGrains` son ambos tipos primitivos, y en Java solo se pueden invocar métodos sobre tipos de referencia, nunca sobre valores primitivos, haciendo que las opciones D y E sean correctas, respectivamente. Por último, la línea 8 compila porque existe un método `length()` definido en la clase `String`.

Pregunta 15

- **Respuesta correcta:** C, E, F
- **Explicación:**

En Java no existen garantías sobre cuándo se ejecutará el recolector de basura (*Garbage Collector*). La JVM es libre de ignorar las llamadas a `System.gc()`. Por este motivo, las opciones A, B y D son incorrectas. La opción C es correcta ya que el propósito de la recolección de basura es recuperar la memoria utilizada. La opción E también es correcta, pues un objeto puede no ser recolectado nunca por el recolector de basura, como cuando el programa finaliza antes de que se ejecute la recolección de basura. La opción F es correcta y representa el criterio principal mediante el cual los algoritmos determinan si un objeto es elegible para ser recolectado. Finalmente, la opción G es incorrecta porque marcar una variable como `final` solo significa que es constante dentro de su propio alcance.

Pregunta 16

- **Respuesta correcta:** A, D
- **Explicación:**

La opción A es correcta. Hay dos líneas: una comienza con `squirrel` y la otra comienza con `pigeon`. Recuerda que una barra invertida (`\`) indica que se debe omitir el salto de línea en esa línea. La opción D también es correcta dado que `\s` indica que se debe conservar el espacio en blanco. En un bloque de texto (*text block*), la sangría incidental se descarta, haciendo que la opción F sea incorrecta.

Pregunta 17

- **Respuesta correcta:** D, F, G
- **Explicación:**

El código compila y se ejecuta sin inconvenientes, por lo que las opciones A y B son incorrectas. Un campo `boolean` se inicializa por defecto en `false`, lo que hace que la opción D sea correcta al imprimirse `Empty = false`. Las referencias a objetos se inicializan en `null` (no en una cadena vacía `""`), de modo que la opción F es correcta al imprimirse `Brand = null`. Finalmente, el valor por defecto de los números de punto flotante es `0.0`. Aunque los valores `float` pueden declararse con un sufijo `f`, no se imprimen con el sufijo `f`. Por estos motivos, la opción G es correcta y se imprime `Code = 0.0`.

Pregunta 18

- **Respuesta correcta:** B, C, F
- **Explicación:**

No se permite el uso de `var` como parámetro de constructores o métodos, ni tampoco para variables de instancia o de clase, lo que invalida la opción A y hace correcta la opción C. El tipo de una variable declarada con `var` se determina en tiempo de compilación y no puede cambiarse durante el tiempo de ejecución, aunque su valor sí puede cambiar en tiempo de ejecución. Por estas razones, las opciones B y F son correctas, y la opción E es incorrecta. La opción D es incorrecta, ya que `var` no está permitido en declaraciones de múltiples variables en una misma

sentencia. Finalmente, la opción G es incorrecta ya que `var` no es una palabra reservada en Java (es un nombre de tipo reservado).

Pregunta 19

- **Respuesta correcta:** A, D
- **Explicación:**

Las dos primeras líneas proporcionan una forma de convertir un `String` en un número. La primera resulta en un primitivo `long` y la segunda en un objeto de tipo de referencia `Long`, lo que hace que la opción D sea una de las respuestas. El código es correcto y el valor máximo alcanzado es `100`, que corresponde a la opción A.

Pregunta 20

- **Respuesta correcta:** C
- **Explicación:**

El punto clave a notar es que la línea 4 no define un constructor, sino un método llamado `PoliceBox()`, dado que cuenta con un tipo de retorno `void`. Este método nunca llega a ejecutarse durante el ciclo del programa, y por ende `color` y `age` reciben sus valores predeterminados: `null` y `0L`, respectivamente. Las líneas 11 y 12 modifican los valores para el objeto apuntado por `p`, pero luego, en la línea 13, la variable `p` se reasigna para apuntar al objeto referenciado por `q`, el cual todavía conserva los valores por defecto. Por esta razón, el programa imprime `Q1=null, Q2=0, P1=null, and P2=0`, convirtiendo a la opción C en la única respuesta correcta.

Pregunta 21

- **Respuesta correcta:** D
- **Explicación:**

Comenzamos con el método `main()`, el cual imprime `7-` en la línea 10. A continuación, se crea una nueva instancia de `Salmon` en la línea 11. Esto provoca que los dos inicializadores de instancia en las líneas 3 y 4 se ejecuten en orden. El valor por defecto de una variable de instancia de tipo `int` es `0`, por lo que a continuación se imprime `0-` y a `count` se le asigna el valor de `1`. Después, se llama al constructor. Este le asigna un valor de `4` a `count` e imprime `2-`. Finalmente, la línea 12 imprime `4-`, ya que ese es el valor de `count`. Juntando todo, tenemos `7-0-2-4-`, haciendo que la opción D sea la respuesta correcta.

Pregunta 22

- **Respuesta correcta:** C, F, G
- **Explicación:**

Primero, `0b` es el prefijo para un valor binario y `0x` es el prefijo para un valor hexadecimal. Estos valores se pueden asignar a muchos tipos primitivos, incluidos `int` y `double`, lo que hace que las opciones C y F sean correctas. La opción A es incorrecta porque nombrar la variable como `Amount` provocará que la llamada a `System.out.print(amount)` en la línea siguiente no compile. La opción B es incorrecta porque `9L` es un valor de tipo `long`. Si el tipo se cambiara a `long amount = 9L`, entonces compilaría. La opción D es incorrecta porque `1_2.0` es un valor de tipo `double`. Si el tipo se cambiara a `double amount = 1_2.0`, entonces compilaría. Las

opciones E y H son incorrectas debido a que el guion bajo (_) aparece justo al lado del punto decimal (.), lo cual no está permitido. Finalmente, la opción G es correcta, y tanto el uso del guion bajo como la asignación son válidos.

Pregunta 23

- **Respuesta correcta:** A, D
- **Explicación:**

El primer error de compilación ocurre en la línea 3. La variable `temp` se declara como un `float`, pero el valor asignado es `50.0`, el cual es un `double` al no incluir el sufijo `F` o `f`. Dado que un `double` no cabe dentro de un `float`, la línea 3 no compila. A continuación, `depth` se declara dentro del bucle `for` y solo tiene alcance dentro de este bucle. Por lo tanto, leer el valor en la línea 10 desencadena un error de compilación. Por estas razones, las opciones A y D son las respuestas correctas.

Capítulo 2 - Operators

Pregunta 1

- **Respuesta correcta:** A, D, G
- **Explicación:**

`==` es el operador de igualdad y se aplica a primitivos y referencias de objetos. `!` es el operador de complemento lógico que se usa exclusivamente con tipos `boolean`. `(boolean)` es un casteo válido puesto que `boolean` es un tipo de dato reconocido en Java. Los demás operadores (`+`, `--`, `%`, `~`) son puramente numéricos o de bits.

Pregunta 2

- **Respuesta correcta:** A, B, D
- **Explicación:**

Por la regla de promoción numérica, al aplicar el operador binario `+` entre tipos enteros menores como `byte` y `short`, ambos se promueven automáticamente a `int`. Por lo tanto, el resultado es un `int` y solo se puede asignar sin casteo explícito a variables de tipo `int`, `long` o `double`.

Pregunta 3

- **Respuesta correcta:** B, C, D, F
- **Explicación:**

La multiplicación `2 * ear` se promueve automáticamente a `long` y no se puede guardar directamente en la variable `hearing` de tipo `int`. Cada opción resuelve el problema de forma independiente: castear `ear` a `int` (B), definir `ear` como `short` (C), castear el resultado `(int)(2 * ear)` (D), o cambiar la variable `hearing` a `long` (F).

Pregunta 4

- **Respuesta correcta:** B
- **Explicación:**

La expresión `(teeth != 10)` evalúa a `true`. La reasignación `(wolf = false)` le otorga el

valor `false` a `wolf` y retorna `false`. Al evaluar `true ^ false` (XOR), el resultado es `true`, el cual se asigna a `canine` mientras que `teeth` no sufre cambios y permanece en `20`.

Pregunta 5

- **Respuesta correcta:** A, C
- **Explicación:**
En la opción A, la adición `+` tiene menor precedencia que los multiplicativos `*` y `%`, los cuales tienen menor o igual precedencia que el operador unario `--`. En la opción C, el operador de asignación `=` tiene la precedencia más baja, seguido del operador de igualdad `==` y la negación unaria `!`.

Pregunta 6

- **Respuesta correcta:** F
- **Explicación:**
Por reglas de precedencia, el casteo (`int`) se aplica de forma aislada a la variable `fruit`. Al sumarse con `vegetables` (de tipo `float`), la expresión completa se promueve a `float`, impidiendo que el método retorne un valor `long` sin un casteo explícito de toda la suma.

Pregunta 7

- **Respuesta correcta:** D
- **Explicación:**
`clear`: `2 > 1 & (true || false)` evalúa a `true`.
`safe`: Como `(vis > 2)` es `false`, el operador `&&` realiza cortocircuito y omite la evaluación de `(ph++ > 1)`. `ph` se mantiene en `7` y `safe` es `false`.
`tasty`: El pre-decremento `--ph` reduce `ph` a `6`. Luego `7 <= 6` evalúa a `false`.

Pregunta 8

- **Respuesta correcta:** A
- **Explicación:**
En `pig = pig++`, al usar el post-incremento, la expresión devuelve el valor original (`4`) para la asignación y la actualización a `5` se descarta. En `goat -= 1.0`, los operadores compuestos aplican un casteo implícito automático al tipo de la variable destino (`long`), reduciendo `goat` de `2` a `1`.

Pregunta 9

- **Respuesta correcta:** A, D, E
- **Explicación:**
Línea 1: `a > 2` (`2 > 2`) es `false`, evalúa `b++` e imprime `4` (D).
Línea 2: `a != c` (`2 != 2`) es `false`, evalúa `b++` (con `b` en `5`), retornando `5` que se asigna nuevamente a `b` (E).
Línea 3: `a > b` (`2 > 5`) es `false`, la rama final del ternario evalúa a `1` (A).

Pregunta 10

- **Respuesta correcta:** G
- **Explicación:**

La línea 2 no compila. Aunque `weight` y `height` se castean individualmente a `byte`, la operación binaria de multiplicación `*` promueve automáticamente el resultado a `int`. Intentar almacenar ese resultado `int` en la variable `short zebra` provoca un error de compilación.

Pregunta 11

- **Respuesta correcta:** D
- **Explicación:**

`sample1: (8) % 3 = 2.`

`sample2: De izquierda a derecha,  $3 * 2 \% 3 = 6 \% 3 = 0$ .`

`sample3:  $5 * (1 \% 2) = 5 * 1 = 5$ .`

Pregunta 12

- **Respuesta correcta:** D
- **Explicación:**

El operador `post-increment` incrementa el valor de la variable pero devuelve el valor original, mientras que el operador `pre-decrement` decrementa la variable y devuelve el valor nuevo ya modificado.

Pregunta 13

- **Respuesta correcta:** F
- **Explicación:**

`goingToTheStore:  $true \& false \wedge true = false \wedge true = true$ .`

`goingToTheZoo:  $true \&\& !false = true$ .`

`stayingHome:  $!(true \&\& true) = false$ .`

Pregunta 14

- **Respuesta correcta:** B, E, G
- **Explicación:**

El operador `!=` se utiliza para comparar referencias entre objetos (B). El valor de retorno de una expresión de asignación es el propio valor reasignado a la variable (E). El complemento lógico `!` actúa únicamente sobre booleanos y no invierte signos numéricos (G).

Pregunta 15

- **Respuesta correcta:** D
- **Explicación:**

El operador ternario (`? :`) es el único operador en Java que requiere exactamente tres operandos para funcionar.

Pregunta 16

- **Respuesta correcta:** B
- **Explicación:**

Únicamente la línea 1 contiene un error de compilación. La expresión `(long)3` promueve la suma entera a `long`, impidiendo que se asigne a la variable `int note` sin un casteo explícito. Las demás líneas utilizan casteos adecuados y asignación compuesta válida.

Pregunta 17

- **Respuesta correcta:** C, F
- **Explicación:**

En `ticketsSold += 1 + ticketsTaken++`, `ticketsTaken++` retorna 1 (y sube a 2), dejando `ticketsSold` en $3 + 1 + 1 = 5$. Luego `ticketsTaken *= 2`; incrementa `ticketsTaken` a 4 (F). Finalmente `ticketsSold += (long)1`; (con casteo implícito en la asignación compuesta) sube `ticketsSold` a 6 (C).

Pregunta 18

- **Respuesta correcta:** C
- **Explicación:**

En Java, únicamente los paréntesis () son caracteres válidos para agrupar expresiones y alterar manualmente la precedencia de los operadores.

Pregunta 19

- **Respuesta correcta:** B, F
- **Explicación:**

En la línea 5, `++start` incrementa `start` a 8 y lo suma a `end` ($4 + 8$), asignando 12 a `end` (F). En la línea 6, `Byte.MAX_VALUE` es 127; al sumarle 1 (128) y castear a `byte`, ocurre un desbordamiento numérico (*overflow*), pasando a -128 (B).

Pregunta 20

- **Respuesta correcta:** A, D, E
- **Explicación:**

Los operadores unarios tienen un nivel de precedencia superior frente a los operadores binarios aritméticos o ternarios (A). El operador post-decremento devuelve el valor previo a la modificación (D). El operador `!` no se puede aplicar a valores numéricos (E).

Pregunta 21

- **Respuesta correcta:** E
- **Explicación:**

`bird` calcula el complemento de bits `~8`, que equivale a $-8 - 1 = -9$. `plane` calcula -8. La comparación `bird == plane` ($-9 == -8$) resulta en `false`, por lo que `superman` recibe el valor 10. Finalmente, `--superman` decrementa la variable antes de imprimir, dando como resultado 9. Imprime -9,-8,9 (E).